

LAB-SDLC-IA-SEC · PONTO 1

Ciclo de vida de desenvolvimento seguro e DevSecOps

Apresentação básica a partir do material do ponto 1

Seis fases · princípios e técnicas · pipeline com segurança

OBJETIVO

O que este ponto ensina

- Entender o SDLC seguro: o que é, as seis fases e as técnicas de cada fase.
- Ver como o DevSecOps executa esse ciclo de forma contínua e automatizada.
- Segurança embutida em cada fase — não um teste no final.

Lógica central: quanto mais cedo a segurança entra, mais barato é corrigir. Uma falha no commit custa horas; a mesma em produção custa dias ou semanas.

OVERVIEW

As seis fases do ciclo

1. Requisitos

Ativos, classificação, CIA, abuse case

2. Design

Least privilege, defense in depth, fail secure, zero trust

3. Implementação

Codificação segura, SAST no commit

4. Verificação

SAST, DAST, SCA, pentest

5. Operação

CVEs e resposta a incidentes

6. Descontinuação

Remoção segura de dados e credenciais

Por que segurança cedo custa menos

- **Commit.** No commit, o código é pequeno e o contexto está fresco.
- **Produção.** A falha já afetou dados, clientes ou receita.
- Corrigir em produção exige investigação, rollback e comunicação.

Exemplo: SQL injection no pull request vira reescrita da query e um teste. A mesma em produção vira incidente, investigação, aviso a clientes e patch emergencial.

Princípios

- **Verificabilidade.** Se o requisito não dá para testar, ele não serve.
- **Rastreabilidade.** Ativo → requisito → controle → teste.
- **Negação explícita.** Abuse case: o que um ator não deve conseguir fazer vira requisito negativo.

A fase responde o que o sistema deve garantir, não como implementar. Técnica central: identificar ativos, classificar dados e escrever requisitos verificáveis.

Técnicas

- **Ativos.** dados, segredos, funções, infra
- **Classificação.** Público, Interno, Confidencial, Restrito
- **CIA.** Confidencialidade, Integridade, Disponibilidade
- **Tipos de requisito.** funcional, não funcional, restrição
- **Critério de aceite.** teste que passa ou falha
- **Premissas.** o que o sistema assume e o que não promete

Para memorizar

- **Técnica.** ativos, classificação, CIA, requisitos verificáveis
- **Princípio.** verificável, rastreável, negação explícita

Regra de ouro: requisito ruim = "seja seguro". Requisito bom = "rota autenticada rejeita token expirado com 401".

Princípios

- **Least privilege.** mínimo de acesso por componente. Pagamentos não lê o banco de usuários.
- **Defense in depth.** várias camadas; uma falha não derruba tudo. WAF + SAST + pentest.
- **Fail secure.** erro nega acesso, nunca libera. O oposto é fail open.
- **Zero trust.** verificar sempre, nunca confiar por padrão — inclusive na rede interna.

Técnicas e regra de ouro

- **Diagrama de fluxo de dados.** base para aplicar os princípios a cada elemento do sistema.
- **Superfície de ataque.** listar pontos de entrada e priorizar por risco.

É a fase mais barata para corrigir erro. Design errado custa uma reunião; código errado custa meses. Técnica central: arquitetura segura com princípios de design.

O resto do ciclo, em uma linha

- **Implementação.** codificação segura. Evitar buffer overflow, SQL injection, funções inseguras. SAST em cada commit.
- **Verificação.** SAST no código, DAST na app rodando, SCA nas dependências, pentest como ataque real.
- **Operação.** rastrear CVEs, responder a incidentes, patch sem derrubar o serviço.
- **Descontinuação.** remover dados, revogar credenciais, transferir de forma controlada.

Development + Security + Operations

- **Development.** o time que escreve o código.
- **Security.** ferramentas e práticas no fluxo, não um time isolado no final.
- **Operations.** deploy, monitoramento e infraestrutura.

Cada commit dispara verificação. Se algo falhar, o pipeline bloqueia o deploy. O oposto é o pentest só no final, quando a correção já está cara.

Shift-left e shift-right

- **Shift-left.** segurança no início. SAST no push, SCA nas dependências, secrets scanning antes do merge.
- **Shift-right.** monitorar em produção. CVE novo aparece todo dia; saber o que afeta o que roda.

Shift-left previne. Shift-right detecta. Um sem o outro deixa buraco: só left ignora o que já está em produção; só right corrige tarde demais.

PIPELINE

CI/CD com segurança

- 1. Commit — o código entra
- 2. Build — compila ou empacota
- 3. Testes — suíte automatizada
- 4. Análise — SAST, SCA, secrets scanning
- 5. Staging — ambiente de teste
- 6. DAST — a app rodando, como um atacante
- 7. Produção — só se tudo passou

Regra de ouro: se uma verificação falhar, o pipeline para. Nada vai para produção com falha de segurança conhecida.

O que cada verificação faz

- **SAST.** código sem executar. Semgrep, SonarQube, CodeQL.
- **DAST.** app rodando. OWASP ZAP, Burp Suite.
- **SCA.** dependências com CVE. Dependabot, Snyk, Trivy.
- **Fuzzing.** entrada aleatória ou malformada; falha de memória e parsing.
- **Secrets.** senha, token, chave no código. gitleaks, trufflehog, GitGuardian.

COMPARAÇÃO

SDLC seguro não é DevSecOps

	SDLC seguro	DevSecOps
Natureza	Processo	Prática
Pergunta	O que fazer?	Como fazer?
Escala	Qualquer projeto	Projetos com CI/CD

Um não substitui o outro. DevSecOps é a forma de executar o SDLC seguro em escala.

Uma linha por fase

- **Requisitos.** ativos, classificação, CIA, abuse case, critério verificável
- **Design.** least privilege, defense in depth, fail secure, zero trust
- **Implementação.** codificação segura
- **Verificação.** SAST, DAST, SCA, pentest
- **Operação.** CVEs e resposta a incidentes
- **Descontinuação.** remoção segura de dados
- **Shift-left / right.** início do ciclo / monitoramento em produção